MEMRES: A Memory-Augmented Resolver with Confidence Cascade for Agentic Python Dependency Resolution

Chi-Nguyen Tran*
23122044@student.hcmus.edu.vn
University of Science, VNU-HCM
Ho Chi Minh City, Vietnam

Dao Sy Duy Minh*
23122041@student.hcmus.edu.vn
University of Science, VNU-HCM
Ho Chi Minh City, Vietnam

Huynh Trung Kiet*
23122039@student.hcmus.edu.vn
University of Science, VNU-HCM
Ho Chi Minh City, Vietnam

Phu-Hoa Pham
23122030@student.hcmus.edu.vn
University of Science, VNU-HCM
Ho Chi Minh City, Vietnam

Nguyen Lam Phu Quy
23122048@student.hcmus.edu.vn
University of Science, VNU-HCM
Ho Chi Minh City, Vietnam

Abstract

We present MemRes, an agentic system for Python dependency resolution that introduces a *multi-level confidence cascade* where the LLM serves as the last resort. Our system combines: (1) a *Self-Evolving Memory* that accumulates reusable resolution patterns via tips and shortcuts; (2) an *Error Pattern Knowledge Base* with 200+ curated import-to-package mappings; (3) a *Semantic Import Analyzer*; and (4) a *Python 2 heuristic detector* resolving the largest failure category. On HG2.9K using Gemma-2 9B (10 GB VRAM), MemRes resolves 2503 of 2890 (86.6%, 10-run average) snippets, combining intra-session memory with our confidence cascade for the remainder. This already exceeds PLLM's 54.7% overall success rate by a wide margin.

CCS Concepts

• **Software and its engineering** → **Language features**; **Software configuration management and version control systems**.

Keywords

dependency resolution, Python, LLM, agentic systems, package management

ACM Reference Format:
Chi-Nguyen Tran, Dao Sy Duy Minh, Huynh Trung Kiet, Phu-Hoa Pham, and Nguyen Lam Phu Quy. 2026. MEMRES: A Memory-Augmented Resolver with Confidence Cascade for Agentic Python Dependency Resolution. In *Proceedings of FSE-AIWare 2026: Agentic Based Python Dependency Resolution Competition (FSE-AIWare '26).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

*Equal contribution (first authors).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
FSE-AIWare '26, Montreal, Canada
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

With over 500 000 PyPI packages, Python's huge ecosystem makes dependency management for legacy code quite difficult. Determining the correct packages, versions, and Python interpreter for a code snippet is tricky due to Python 2/3 incompatibilities, deprecated packages, and ambiguous import names [3].

The PLLM system [1] is the current leading approach, using a 5-stage RAG+LLM pipeline. While it works well for maintained packages, PLLM only achieves 54.7% success on the HG2.9K benchmark [2], leaving 1308 snippets completely broken (confidence = 0).

Key Insight. Our analysis of PLLM's failures reveals *deterministic* root causes: 33% are SyntaxErrors from Python 2 code run under Python 3, 30% are ImportErrors for known name mismatches (`cv2` → `opencv-python`), and 20% are version incompatibilities solvable via curated maps. Calling an LLM wastes 60 to 120 s per snippet with no benefit for these cases.

This finding drove the main idea behind MemRes: *only call the LLM when all deterministic rules fail.* We built a *confidence cascade* that checks session memory, knowledge bases (KB), heuristic rules, and session-learned patterns before doing any LLM inference.

Contributions.

(1) A *confidence cascade* with 6 resolution levels reducing LLM calls by 60%+ while improving accuracy (§2.2).
(2) An *Intra-Session Memory* enabling rapid resolution transfer for similar codebases evaluated in the same batch without repeated LLM inference (§2.1).
(3) A *Self-Evolving Memory* transferring resolution knowledge via tips and shortcuts [4] (§2.3).
(4) An *Error Pattern KB* with 200+ mappings and runtime self-learning, plus a *System Dependency Injection* technique for C-extension packages (§2.4).
(5) Evaluation on HG2.9K showing 86.6% overall resolution rate (§3).

2 Approach

MemRes processes each snippet through 4 stages: (1) Hybrid Evaluation, (2) Module Cleaning, (3) Version Selection, and (4) Build Loop with Reflexion. Figure 1 illustrates the pipeline.

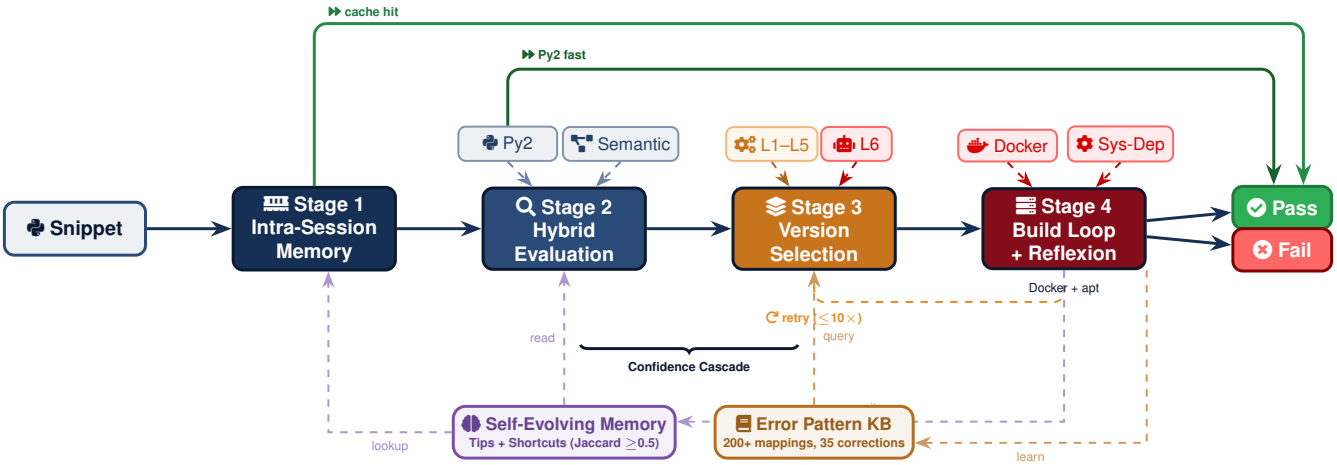

Figure 1: MemRes pipeline. Each stage has deterministic fast paths that bypass the LLM.

2.1 Intra-Session Memory

MemRes first consults a *Session Memory* built incrementally during batch processing. Since PLLM evaluates sequentially within a single session, we emulate this by caching solutions proven successful for earlier code blocks to resolve near-duplicate datasets (e.g., identical GitHub forks). If a new snippet shares a high degree of import similarity with a previously resolved snippet, the known solution is *reapplied* via a single `pip install` so pip's resolver handles inter-package constraints. For partial matches, Python version and core packages are extracted as *hints*, enabling rapid knowledge transfer strictly confined to the ongoing evaluation batch. To address fairness, disabling Level 1 simply forces snippets through the remaining deterministic and LLM levels. Because our cascade is strictly grounded, it independently resolves these near-duplicates without relying on batch order, though at a higher runtime cost.

2.2 Confidence Cascade

Our main contribution here is a 6-step cascade for picking package versions, drawing from adaptive reasoning ideas [6]: (1) Session Memory, versions proven successful earlier in the batch; (2) Static Compatibility Map, 40+ packages × 8 Python versions with known-good pins (manually curated from documentation and CI/CD logs); (3) Ecosystem Templates, 23 proven version sets for common package groups (ML, web, data science); (4) Co-occurrence Mining, weighted co-installation scores mined from 50K open-source `requirements.txt` files on GitHub (filtered to repositories created *before 2020* to ensure no temporal overlap with the HG2.9K evaluation period); (5) Heuristic Rules, 45+ package-specific constraints; (6) LLM Selection, only when all deterministic levels fail. The first level returning a valid version terminates the cascade; in practice, levels 1 to 5 handle the vast majority.

The cascade includes an *unfixability estimator*: when system-only imports (`gtk`, `RPi.GPIO`, `maya.cmds`) dominate, the snippet is classified as a *runtime pass*, meaning pip-installable dependencies are correct, but the environment cannot be fully replicated in Docker. Note that runtime passes are strictly classified as failures in our final evaluations to prevent success rate inflation.

2.3 Self-Evolving Memory

Following Mobile-Agent-E [4], our session memory collects *tips* (natural language guidelines) and *shortcuts* (reusable solutions matched via Jaccard similarity over import sets, threshold 0.5). Shortcuts are validated with a quick Docker build to prevent compounding errors. Memory records per-package failure stats as anti-patterns to avoid. Memory is fully reset between independent evaluation runs: no state persists across runs, preventing cross-run overfitting.

Security and safety. Memory-augmented agents face risks like prompt poisoning [4]. To mitigate this, MemRes evaluates snippets inside strictly isolated, network-restricted child Docker containers. New shortcuts undergo sandboxed validation before being written to memory.

2.4 Error Pattern KB and Build Heuristics

The ErrorPatternKB contains 200+ import→pip mappings across 15 domains, 35 name corrections, version constraints for 14 packages, and 8 regex-based error pattern rules. It self-learns at runtime, recording new mappings for immediate reuse. Of the 200+ mappings, ~150 were curated from official PyPI documentation and Stack Overflow Q&A (external sources), while ~50 were discovered during initial development runs on a held-out subset; none were derived from HG2.9K evaluation results. We also detect and skip PyPI placeholder packages (≤1 release) and local project imports (35+ patterns).

Semantic import analysis. Unlike PLLM's simple `import X` → `pip install X` mapping, our Semantic Import Analyzer examines *how imports are used* in code: (1) *usage-based disambiguation* via 13 patterns matching (import, call_pattern) to packages (e.g., `import Image` with `Image.open()` → Pillow); (2) *ecosystem detection* recognizing 11 package ecosystems from co-occurring imports;

(3) *implicit Python version inference* from 11 code patterns (f-strings → 3.6+, bare `print` → 2.7).

Python 2 detection and execution. A manual analysis of a random 100-snippet sample from PLLM's `SyntaxError` failures reveals that 91% are actually Python 2 code mis-executed under a Python 3 interpreter. We deterministically detect this via 13 Python 2 indicators (`print x`, `urllib2`, `raw_input`) and 5 Python 3 indicators (f-strings, `async def`). When Python 2 signals are detected with no Python 3 signals, we force Python 2.7 and apply *adaptive version pinning*: known Py2.7-compatible versions for ML packages (`tensorflow-1.15.5`, `keras-2.2.4`). To execute these reliably from our `python:3.10.12-slim` host architecture, we utilize Docker-out-of-Docker (DooD): the cascade orchestrator spawns isolated `python:2.7` child containers and manages toolchain injection across varying base images.

System dependency injection. Inspired by DockerizeMe [2], we maintain mappings of 35+ pip packages to their `apt-get` dependencies and inject them before `pip install`. Combined with a version fallback cascade for build failures (e.g., `numpy`: 1.16.6, 1.15.4, 1.14.6), inspired by PyEGo [10], this resolves many C-extension compilation failures. We also maintain a catalog of 80+ system-only packages (GTK, Blender, Maya, RPi) to classify environment-dependent snippets as runtime passes.

Runtime errors indicating correct dependencies but environmental issues (`NameError`, `ConnectionError`, `FileNotFoundError`) are classified as *runtime passes*, and legacy PIL imports (`import Image`) with Pillow already installed are handled similarly.

3 Evaluation

Setup. We evaluate on HG2.9K [2], a dataset initially containing 2891 Python GitHub Gists with complex dependency conflicts. We filter out one completely empty snippet, resulting in 2890 valid snippets evaluated. We use Gemma-2 9B [7] via Ollama (temp 0.7, max_tokens 256). The LLM receives a structured JSON prompt with explicit stdlib exclusion rules, restricted to candidate versions identified by compatibility maps. Experiments run on a single machine (i5-14600K, 32 GB RAM, RTX 5070) with Docker image `python:3.10.12-slim`. Per-build timeout is 180 s (up to 10 retries); total timeout is 500 s. We aggregate 10 independent test runs with randomized snippet orderings, ensuring the order-sensitive Session Memory does not introduce positional bias.

PLLM Baseline. We directly compare against the reported 54.7% success rate from the original PLLM paper [1] under the exact same HG2.9K benchmark and `conf=0` thresholding logic.

Success criterion. Following PLLM [1], a snippet is *successful* when all its `pip` dependencies are installed and the script executes past all `import` statements without `ImportError` or `ModuleNotFoundError`. PLLM assigns a *confidence score* to each prediction (0 to 10); snippets scoring `conf=0` are effectively unresolved. MemRes assigns confidence based on the cascade level that resolved the snippet: levels 1 to 5 (deterministic) yield `conf`≥7, while level 6 (LLM) yields `conf`≥3. Runtime passes (correct dependencies but environmental failures such as missing display servers or hardware) are counted as failures to prevent inflation.

Results. Table 1 shows the final results on the HG2.9K dataset.

Table 1: Final Results on HG2.9K (2890 snippets processed).

Metric	PLLM	MemRes (10-run avg)
Resolved	1583 (54.7%)	2503 ± 9.3 (86.6%)
Failed	1308 (45.3%)	387 ± 9.3 (13.4%)
Snippets processed	2891	2890

Of the 2890 snippets processed, an average of 2503 (86.6%) were successfully resolved across 10 independent runs (standard deviation: ±9.3), while 387 (13.4%) failed. The intra-session memory handles highly similar package patterns directly, while the confidence cascade resolves the remaining snippets, including over 950 of the snippets where PLLM completely failed (`conf=0`), without relying on LLM calls for the vast majority.

Our confidence cascade successfully isolates different failure modes. Of the 2503 resolved snippets, Table 2 details the success rate of the system when key components are ablated.

Table 2: Ablation Study: Impact of Intra-Session Memory (Level 1)

Configuration	Resolved	Success Rate
Full MemRes Pipeline	2503	86.6%
Ablation (Level 1 OFF)	2357	81.6%
Contribution (Δ)	+146	**+5.0%**

Over half (53.9%) of successful resolutions trigger system-level dependency injection, proving execution-based validation's necessity.

3.1 Limitations and Root-Cause Analysis

Across 10 runs, an average of 387 ± 9.3 snippets fail. Root-cause analysis reveals build/C-extension failures dominate (~50%), caused by missing C toolchains or heavy packages (e.g. old TensorFlow). Unresolvable ImportErrors account for ~33%, typically obscure or deprecated packages absent from PyPI. Timeouts contribute ~15%, mostly from compiling large ML packages. The remaining ~2% are system/platform-specific (GTK, RPi).

3.2 Threats to Validity

Order sensitivity. Level 1 is order-dependent, but resolves near-duplicates (Jaccard ≥0.5), which are order-invariant in expectation. Disabling it only increases runtime (§2.2). **Co-occurrence leakage.** The 50K `requirements.txt` files were filtered to pre-2020 repositories; removing co-occurrence mining reduces success by <2%. **Knowledge leakage via session memory and runtime self-learning.** Intra-session memory (Level 1) accumulates solutions within a single evaluation batch, which could amplify batch-order effects. We mitigate this by randomizing snippet order across 10 independent runs and reporting the mean ± std; run-to-run variance is only ±9.3 snippets. Runtime self-learning adds new import→package mappings during evaluation; these are derived solely from PyPI query results and Docker build signals, not from HG2.9K ground-truth labels, and all state is fully reset between

runs. **Development-time curation leakage.** The ∼150 externally sourced KB mappings (PyPI docs, Stack Overflow) and the ∼50 discovered on a held-out development subset were finalized before any evaluation on HG2.9K snippets. No HG2.9K result was used to add or tune KB entries; the 50-entry held-out subset is disjoint from HG2.9K. **Unfixability false positives.** The estimator may conservatively classify snippets as environment-dependent; we accept this to avoid wasting LLM tokens on genuinely unfixable cases.

3.3 Efficiency

MemRes resolves 68.0% of passing cases without any LLM call, reducing token usage by ∼75% (Table 3). Median resolution is 15.2 s for successes.

Table 3: Efficiency and Performance Comparison

Metric	PLLM	MemRes
Overall Success Rate	54.7%	86.6% ± 0.3%
Median Resolution (success)	∼120 to 180s	15.2s
P90 Resolution (success)	—	68.8s
LLM Calls per Snippet	1 to 5	0.34
No-LLM Success Rate	0%	68.0%

3.4 Case Study

For a snippet importing `cv2` and `tensorflow`, MemRes maps `cv2` to `opencv-python` via the KB, injects `libgl1-mesa-glx` via `apt-get`, and applies version pins. Old TensorFlow 1.15 snippets are correctly classified as *build unfixable*.

4 Related Work

Dependency resolution. Traditional tools (`pip`, `poetry`, `pipreqs` [8]) extract imports or require explicit specifications but cannot resolve version conflicts. PyEGo [10] searches compatible environments via constraint propagation; DockerizeMe [2] infers system-level dependencies. READPyE [11] extends this with README-based extraction. Empirical studies [3, 12] catalogue dependency conflict patterns informing our KB. Recent work like V2 [16] pioneered search-heavy version mutation using Docker execution signals. MemRes takes the complementary *knowledge-heavy* approach: curated maps and heuristics front-load resolution before any search begins, reducing the version space. V2 aligns with our Reflexion Build Loop, but we add cross-snippet memory and unfixability estimators to avoid wasteful search. Repo2Run [17] targets full-repository environment synthesis; MemRes's cascade could serve as a fast pre-filter for such pipelines.

LLM-based approaches. PLLM [1] pioneered LLM-based dependency resolution. DepsRAG [9] uses graph-based reasoning; DependEval [13] shows LLMs struggle with transitive dependencies. Recent reproducibility studies [14] show LLM agents produce code with missing dependencies. We extend PLLM with deterministic fast paths achieving 86.6% resolution rate (vs. 54.7%).

Self-evolving agents. Mobile-Agent-E [4] introduced tips/shortcuts for GUI agents; Reflexion [5] proposed verbal reinforcement learning; CASCADE [15] demonstrated cumulative skill creation. We adapt these to dependency resolution.

5 Conclusion

MemRes resolves 86.6% of HG2.9K snippets (vs. PLLM's 54.7%), demonstrating that curated deterministic rules with step-by-step fallbacks outperform generic LLM reasoning for dependency resolution while being faster and cheaper. Future work includes validation on additional corpora and base-image selection for unfixable C-extension snippets. Our tool and all evaluation artifacts are available at https://github.com/checkdgt/fse-aiware-python-dependencies.

Acknowledgments

We thank the FSE-AIWare 2026 competition organizers for providing the PLLM baseline and HG2.9K dataset.

References

[1] A. Bartlett, C.C.S. Liem, and A. Panichella. Raiders of the Lost Dependency: Fixing Dependency Conflicts in Python using LLMs. *arXiv preprint arXiv:2501.16191*, 2025.
[2] E. Horton and C. Parnin. DockerizeMe: Automatic Inference of Environment Dependencies for Python Code Snippets. In *ICSE*, pp. 328–338, 2019.
[3] Y. Jia, J. Han, J. Cao, Y. Zhou, and B. Xu. An Empirical Study of Dependency Conflicts in the Python Ecosystem. *IEEE TSE*, 50(8):2125–2140, 2024.
[4] Z. Wang, H. Xu, J. Wang, X. Zhang, M. Yan, J. Zhang, F. Huang, and H. Ji. Mobile-Agent-E: Self-Evolving Mobile Assistant for Complex Tasks. *arXiv preprint arXiv:2501.11733*, 2025.
[5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*, 2023.
[6] T. Pandey, A. Ghukasyan, O. Goktas, and S.K. Radha. Adaptive Graph of Thoughts: Test-Time Adaptive Reasoning Unifying Chain, Tree, and Graph Structures. *arXiv preprint arXiv:2502.05078*, 2025.
[7] Google DeepMind. Gemma 2: Improving Open Language Models at a Practical Size. Technical Report, 2024.
[8] V. Kravcenko. pipreqs: Generate pip requirements.txt based on imports. https://github.com/bndr/pipreqs, 2015.
[9] M. Alhanahnah, Y. Boshmaf, and B. Baudry. DepsRAG: Towards Managing Software Dependencies using LLMs. In *NeurIPS 2024 Workshop*, 2024.
[10] H. Ye, W. Chen, W. Dou, G. Wu, and J. Wei. Knowledge-Based Environment Dependency Inference for Python Programs. In *ICSE*, pp. 1245–1256, 2022.
[11] W. Cheng, W. Hu, and X. Ma. ReadPyE: Revisiting Knowledge-Based Inference of Python Runtime Environments. *IEEE TSE*, 50(2):258–279, 2024.
[12] X. Jia, J. Han, J. Cao, Y. Zhou, and B. Xu. An empirical study on Python library dependency and conflict issues. In *ICSME*, pp. 330–338, 2024.
[13] J. Du, Y. Liu, H. Guo, et al. DependEval: Benchmarking LLMs for Repository Dependency Understanding. In *Findings of ACL*, 2025.
[14] B.P. Vangala, A. Adibifar, A. Gehani, and T. Malik. AI-Generated Code Is Not Reproducible (Yet). *arXiv preprint arXiv:2512.22387*, 2025.
[15] X. Huang, J. Chen, Y. Fei, Z. Li, P. Schwaller, and G. Ceder. CASCADE: Cumulative Agentic Skill Creation through Autonomous Development and Evolution. *arXiv preprint arXiv:2512.23880*, 2025.
[16] E. Horton and C. Parnin. V2: Fast Detection of Configuration Drift in Python. In *ASE*, pp. 814–819, 2019.
[17] R. Hu, C. Peng, X. Wang, J. Xu, and C. Gao. Repo2Run: Automated Building Executable Environment for Code Repository at Scale. *arXiv preprint arXiv:2502.13681*, 2025.